Task 14: Microservices-based system

Aim: To design a microservices-based system by splitting functionalities into independent logical services within a single Node.js application.

Algorithm:

1. Initialize Express server
2. Create User module (CRUD operations)
3. Create Order module (CRUD operations)
4. Validate inputs before processing
5. Link Order with User (simulate service communication)
6. Handle errors globally
7. Test all APIs using Postman

Code:

Server.js

```
const express = require('express');

const app = express();


app.use(express.json());


// ------------------ HOME ROUTE (FIX ADDED) ------------------

app.get('/', (req, res) => {

   res.send("Microservices API is running 🚀 ");

});


// ------------------ USER SERVICE ------------------

let users = [];


// CREATE USER

app.post('/users', (req, res) => {

   const { id, name } = req.body;


   if (!id || !name) {
```

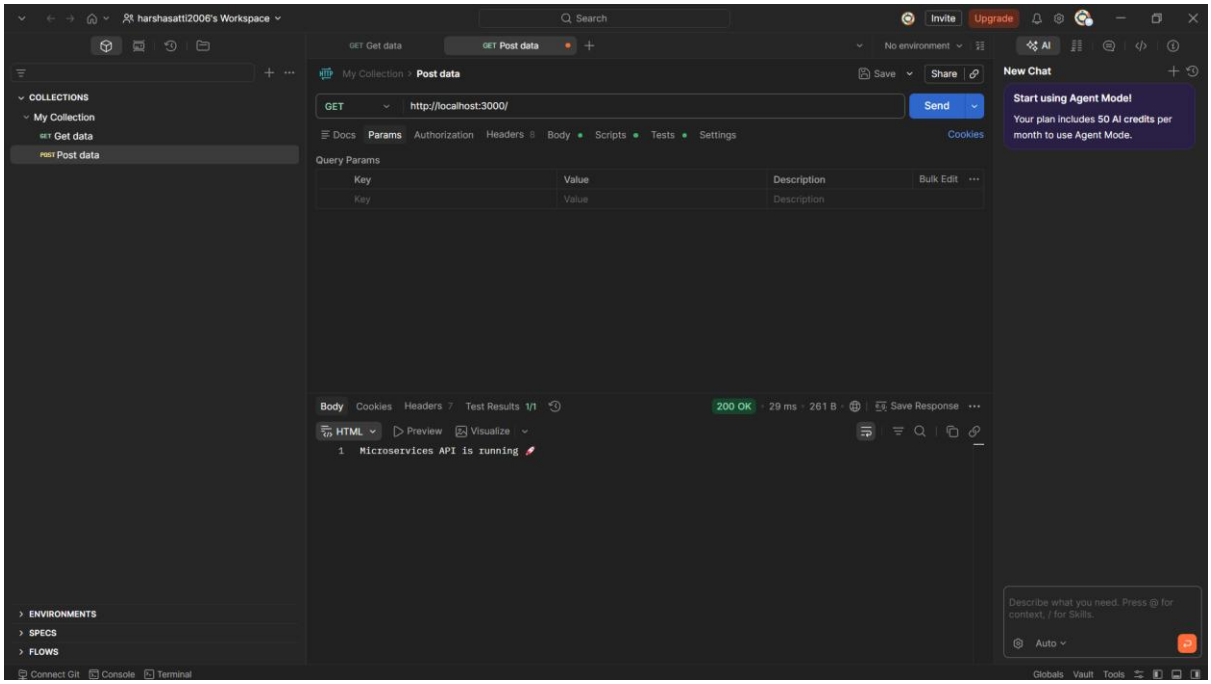
```
        return res.status(400).json({ error: "Invalid input: id and name required" });
    }

    const exists = users.find(u => u.id == id);
    if (exists) {
        return res.status(400).json({ error: "User already exists" });
    }

    users.push({ id, name });
    res.json({ message: "User added successfully", users });
});

// GET ALL USERS
app.get('/users', (req, res) => {
    res.json(users);
});

// GET USER BY ID
app.get('/users/:id', (req, res) => {
    const user = users.find(u => u.id == req.params.id);
    if (!user) return res.status(404).json({ error: "User not found" });

    res.json(user);
});

// UPDATE USER
app.put('/users/:id', (req, res) => {
    const user = users.find(u => u.id == req.params.id);
    if (!user) return res.status(404).json({ error: "User not found" });
```

```
    user.name = req.body.name || user.name;
    res.json({ message: "User updated successfully", user });
});

// DELETE USER
app.delete('/users/:id', (req, res) => {
    const exists = users.find(u => u.id == req.params.id);
    if (!exists) return res.status(404).json({ error: "User not found" });

    users = users.filter(u => u.id != req.params.id);
    res.json({ message: "User deleted successfully" });
});

// ------------------ ORDER SERVICE ------------------
let orders = [];

// CREATE ORDER
app.post('/orders', (req, res) => {
    const { id, userId, product } = req.body;

    if (!id || !userId || !product) {
        return res.status(400).json({ error: "Invalid input: id, userId, product required" });
    }

    const user = users.find(u => u.id == userId);
    if (!user) {
        return res.status(404).json({ error: "User not found (User Service)" });
    }
```

```
    const exists = orders.find(o => o.id == id);
    if (exists) {
        return res.status(400).json({ error: "Order already exists" });
    }

    const order = { id, userId, product };
    orders.push(order);

    res.json({
        message: "Order created successfully",
        order,
        user
    });
});

// GET ALL ORDERS
app.get('/orders', (req, res) => {
    res.json(orders);
});

// GET ORDER BY ID
app.get('/orders/:id', (req, res) => {
    const order = orders.find(o => o.id == req.params.id);
    if (!order) return res.status(404).json({ error: "Order not found" });

    res.json(order);
});
```

```
// UPDATE ORDER
app.put('/orders/:id', (req, res) => {
    const order = orders.find(o => o.id == req.params.id);
    if (!order) return res.status(404).json({ error: "Order not found" });

    order.product = req.body.product || order.product;
    res.json({ message: "Order updated successfully", order });
});

// DELETE ORDER
app.delete('/orders/:id', (req, res) => {
    const exists = orders.find(o => o.id == req.params.id);
    if (!exists) return res.status(404).json({ error: "Order not found" });

    orders = orders.filter(o => o.id != req.params.id);
    res.json({ message: "Order deleted successfully" });
});

// ------------------ GLOBAL ERROR HANDLER ------------------
app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(500).json({ error: "Something went wrong!" });
});

// ------------------ START SERVER ------------------
app.listen(3000, () => {
    console.log("Server running on http://localhost:3000 🚀 ");
});
```

Outputs:



Result: A microservices-inspired system was implemented in a single file. All CRUD operations for User and Order services were successfully tested and passed in Postman.